



INTRODUCTION TO LINEAR ALGEBRA

MATRICES, VECTORS, AND DOT PRODUCTS

LEARNING OUTCOMES

- Understand how and why linear algebra is useful for **analyzing data**
- Learn how to think about **high-dimensional vectors** and their geometry
- Grasp the intuition and mechanics of **matrix-vector and matrix-matrix multiplication**
- Learn how to perform elementary linear algebra operations with **numpy**

Note: this learning unit contains mostly mathematical theory, along with practical examples for elementary linear algebra computations with **numpy**.

WHAT IS LINEAR ALGEBRA?

Linear algebra is the study of techniques for manipulating and characterizing vectors.

An M -dimensional vector is a collection of M real numbers

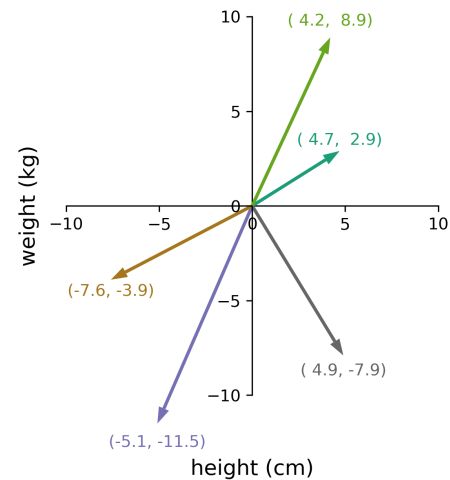
$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_M \end{bmatrix} \in \mathbb{R}^M$$

WHY LINEAR ALGEBRA?

- Multivariate data can be thought of as a collection of vectors: a data set with N data samples with M features is just a collection of N M -dimensional vectors!
- Viewed in this way, **analyzing data** simply boils down to linear algebra: **manipulating and characterizing vectors**

Example: heights and weights of various athletes can be seen as a collection of two-dimensional vectors containing the height and weight of each athlete

	height (cm)	weight (kg)
0	4.7	2.9
1	-5.1	-11.5
2	4.2	8.9
3	-7.6	-3.9
4	4.9	-7.9



CHARACTERIZING VECTORS

VECTOR NORM

HOW "BIG" IS A VECTOR?

- L^2 norm: use Pythagoras theorem!

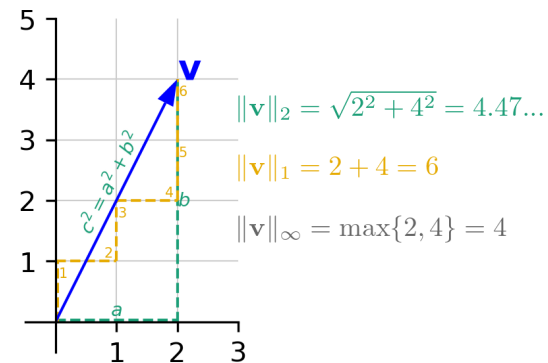
$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_M^2}$$

- L^1 norm: how many "blocks" do you have to travel? (the "Manhattan norm")

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \dots + |v_M|$$

- L^∞ norm: largest element of the vector

$$\|\mathbf{v}\|_\infty = \max |v_1|, |v_2|, \dots, |v_M|$$

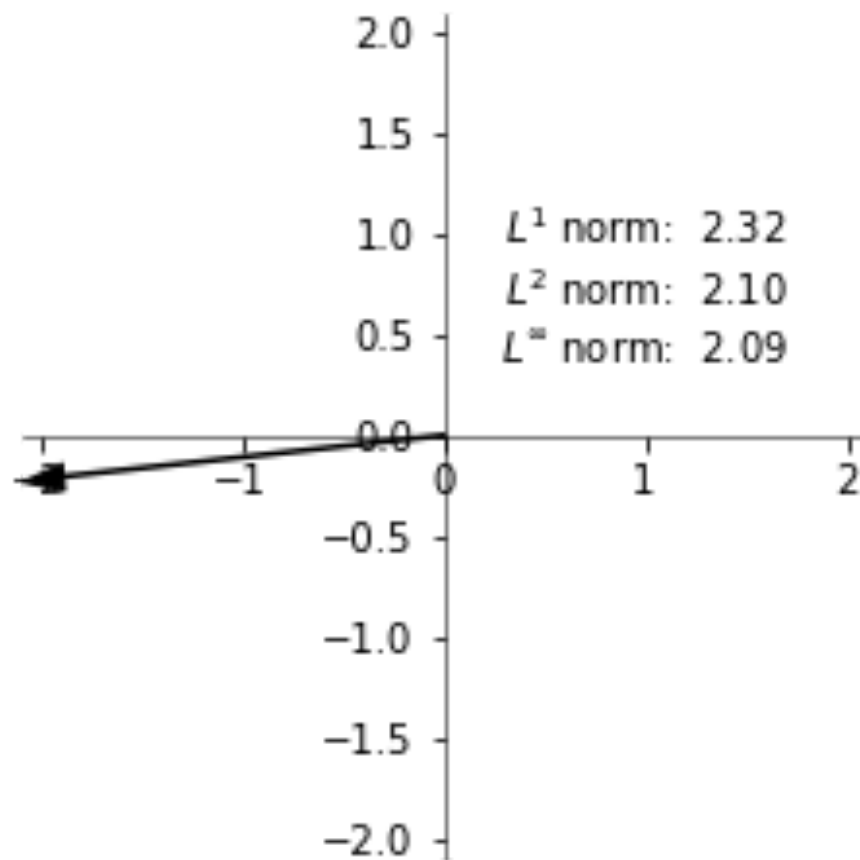


VECTOR NORMS IN numpy

- use `linalg.norm()` function
- pick which norm using `ord` argument

```
v = np.random.randn(2)           # a random 2D vector
norm_L1 = np.linalg.norm(v, ord=1) # calculate L1 norm
norm_L2 = np.linalg.norm(v, ord=2) # calculate L2 norm
norm_Linf = np.linalg.norm(v, ord=np.inf) # calculate L-infinity norm
```

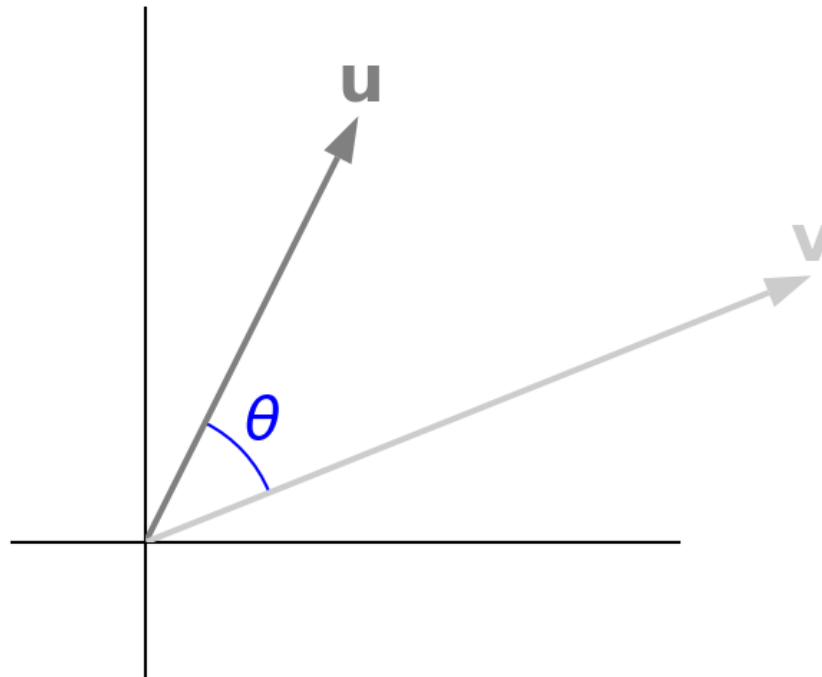
VECTOR NORMS IN `numpy`



VECTOR ANGLE

WHAT IS THE "DIRECTION" OF A VECTOR?

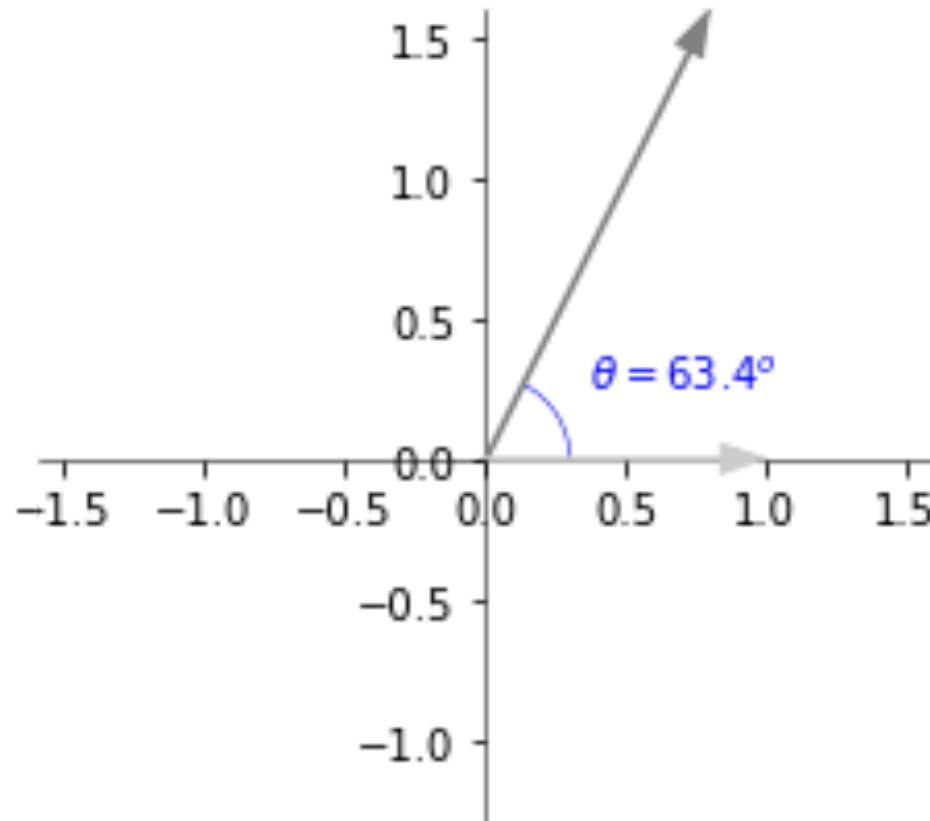
- E.g. 2D vector pointing "up" = 90° angle from horizontal axis
- For higher-dimensional vectors, "up" is meaningless, but can still measure the angle between any two vectors!
- Two vectors are **orthogonal** if their angle is 90°



VECTOR ANGLE

WHAT IS THE "DIRECTION" OF A VECTOR?

```
# Pick two vectors  
u = np.array([0.8, 1.6])  
v = np.array([1, 0])  
  
# Plot the two vectors with their angle  
draw_vector_pair(u, v)
```



DOT PRODUCT

$$\begin{aligned}\mathbf{u} \cdot \mathbf{v} &= \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix} = \sum_{i=1}^N u_i v_i \\ &= \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta\end{aligned}$$

- Measures "overlap" between two vectors (see appendix)
- If $\mathbf{u}$ and $\mathbf{v}$ are orthogonal, then $\mathbf{u} \cdot \mathbf{v} = 0$

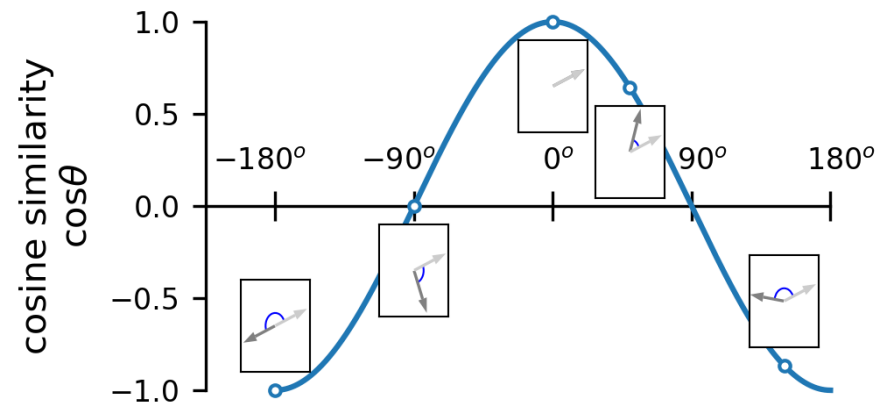
DOT PRODUCT

```
dotprod = np.dot(u, v)  # np.dot() computes dot product
```

COSINE SIMILARITY

HOW "SIMILAR" ARE TWO VECTORS?

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$



COSINE SIMILARITY IN numpy

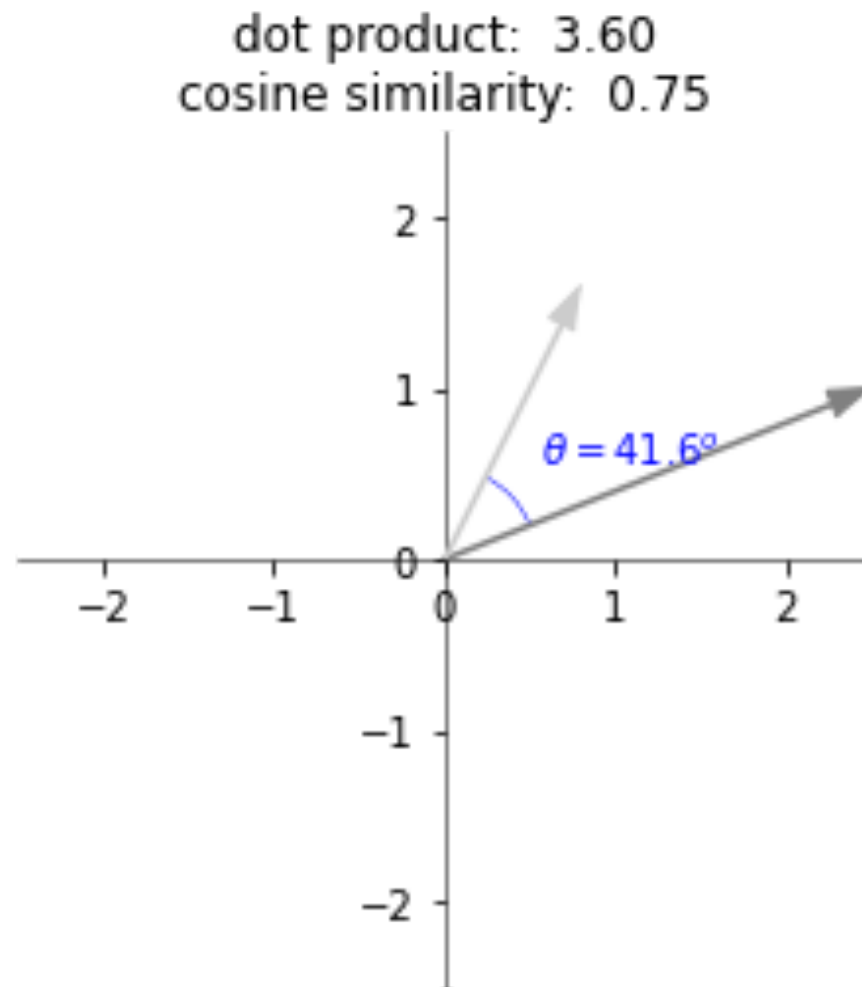
```
# Calculate dot product
```

```
dotprod = np.dot(u, v)
```

```
# Calculate cosine similarity
```

```
cossim = np.dot(u, v) / (np.linalg.norm(u, ord=2) * np.linalg.norm(v, ord=2))
```

COSINE SIMILARITY IN `numpy`

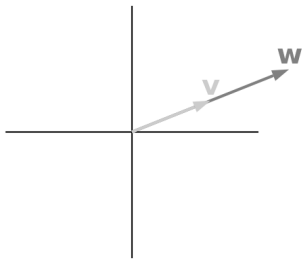


MANIPULATING VECTORS

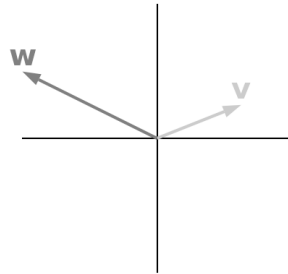
LINEAR INDEPENDENCE

A set of vectors are **linearly independent** if none of them can be produced from linear combinations of the others

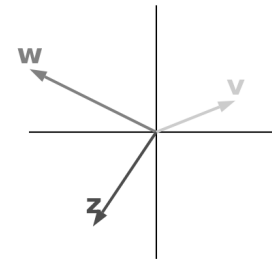
not linearly independent



linearly **independent**



not linearly independent



LINEAR INDEPENDENCE

If $\mathbf{u}$ and $\mathbf{v}$ are linearly independent then any 2D vector can be formed by a linear combination of $\mathbf{u}$ and $\mathbf{v}$!

- In M dimensions, up to M vectors can be linearly independent.
- M linearly independent M -dimensional vectors form a **complete basis**: any M -dimensional vector can be produced by a linear combination of these vectors

MATRICES

WHAT IS A MATRIX?

An $M \times N$ array of numbers

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \vdots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix}$$

- M is the number of rows, N is the number of columns
- A_{ij} is the element of the matrix $\mathbf{A}$ in the i th row and j th column
- If $M = N$, it is called a **square** matrix, and has some special properties

MATRIX TRANSPOSE

The **transpose** of an $N \times M$ matrix is obtained by swapping its rows/columns, yielding an $M \times N$ matrix:

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \vdots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix} \rightarrow \mathbf{A}^T = \begin{bmatrix} A_{11} & A_{21} & \dots & A_{M1} \\ A_{12} & A_{22} & \dots & A_{M2} \\ \vdots & \vdots & \vdots & \vdots \\ A_{1N} & A_{2N} & \dots & A_{MN} \end{bmatrix}$$

In `numpy`:

```
print('original matrix:\n', A)
print('matrix transpose:\n', A.T)
```

```
original matrix:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
matrix transpose:
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

MATRIX-VECTOR MULTIPLICATION

Multiplying a matrix with a vector results in another vector:

$$\mathbf{A}\mathbf{v} = \underbrace{\begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix}}_{(M \times N)} \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix}}_{(N \times 1)} = \underbrace{\begin{bmatrix} \sum_{i=1}^N A_{1i}v_i \\ \sum_{i=1}^N A_{2i}v_i \\ \vdots \\ \sum_{i=1}^N A_{Mi}v_i \end{bmatrix}}_{(M \times 1)}$$

- **matrix-vector multiplication is not always allowed!** $\mathbf{A}$ and $\mathbf{v}$ can only be multiplied if $\mathbf{v}$ has as many components as $\mathbf{A}$ has columns

```
Av = np.dot(A, v)           # can use np.dot() for matrix-vector multiplication
print(A.shape, "*", v.shape, "=", Av.shape)
```

$(4, 10) * (10,) = (4,)$

INTERPRETATION #1:

MATRICES AS COLUMNS

- compute linear combination of matrix columns
- can think of $\mathbf{v}$ as "weights" weighting each column in the combination

$$\begin{aligned} \mathbf{A}\mathbf{v} &= \begin{bmatrix} A_{11} \\ A_{21} \\ \vdots \\ A_{M1} \end{bmatrix} + \begin{bmatrix} A_{12} \\ A_{22} \\ \vdots \\ A_{M2} \end{bmatrix} + \dots + \begin{bmatrix} A_{1N} \\ A_{2N} \\ \vdots \\ A_{MN} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix} \\ &= v_1 \begin{bmatrix} A_{11} \\ A_{21} \\ \vdots \\ A_{M1} \end{bmatrix} + v_2 \begin{bmatrix} A_{12} \\ A_{22} \\ \vdots \\ A_{M2} \end{bmatrix} + \dots + v_N \begin{bmatrix} A_{1N} \\ A_{2N} \\ \vdots \\ A_{MN} \end{bmatrix} \end{aligned}$$

```
Av = np.dot(A, v) # matrix-vector multiplication with np.dot()

m, n = A.shape

Av_cols = np.zeros(m)

for i, vi in enumerate(v):
    Av_cols += vi * A[:, i] # linearly combine columns

print('using dot: ', Av)
print('using linear combination of columns: ', Av_cols)
```

using dot: [14 38 62]

using linear combination of columns: [14. 38. 62.]

INTERPRETATION #2:

MATRICES AS ROWS

- compute dot products between the vector and each row of the matrix
- can think of $\mathbf{v}$ as a "template" that each row gets compared to

$$\mathbf{A}\mathbf{v} = \begin{bmatrix} \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \end{bmatrix} \\ \begin{bmatrix} A_{21} & A_{22} & \dots & A_{2N} \end{bmatrix} \\ \vdots \\ \begin{bmatrix} A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix} \end{bmatrix} \mathbf{v}$$
$$= \begin{bmatrix} \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \end{bmatrix} \cdot \mathbf{v} \\ \begin{bmatrix} A_{21} & A_{22} & \dots & A_{2N} \end{bmatrix} \cdot \mathbf{v} \\ \vdots \\ \begin{bmatrix} A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix} \cdot \mathbf{v} \end{bmatrix}$$

```
Av = np.dot(A, v)    # matrix-vector multiplication with np.dot()

m, n = A.shape

Av_rows = np.zeros(m)

for i in range(m):
    Av_rows[i] = np.dot(v, A[i, :]) # dot products with each row

print('using dot: ', Av)
print('using dot products with rows: ', Av_rows)
```

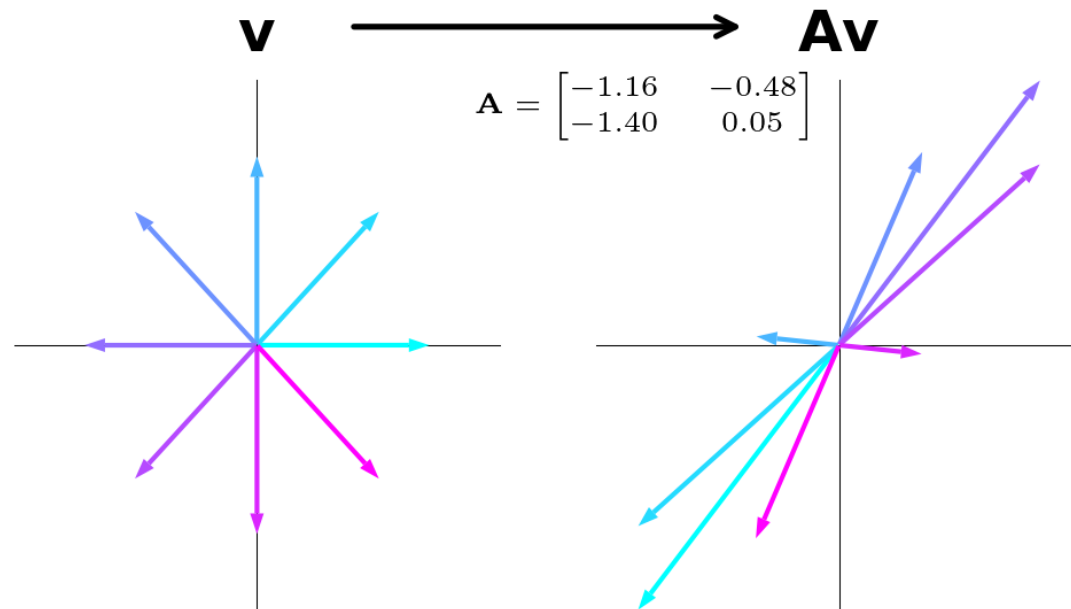
using dot: [14 38 62]

using dot products with rows: [14. 38. 62.]

INTERPRETATION #3 [FOR SQUARE MATRICES ONLY]:

MATRICES AS TRANSFORMATIONS

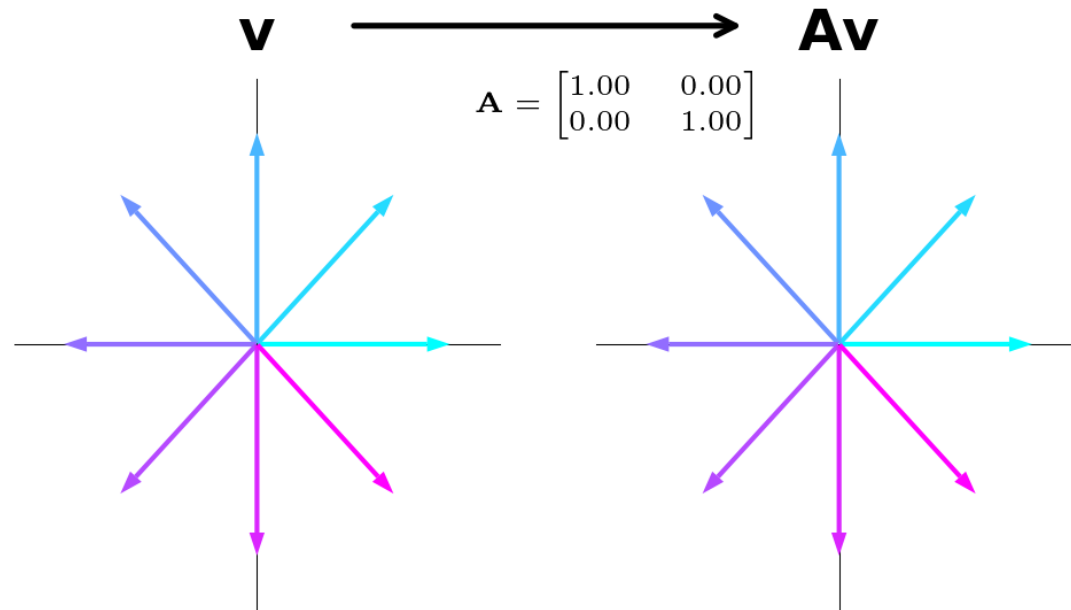
An $M \times M$ square matrix transforms an M -dimensional vector into another one



SPECIAL SQUARE MATRICES:

IDENTITY MATRIX

The simplest possible transformation: transforming $\mathbf{v}$ into itself, i.e. $\mathbf{A}\mathbf{v} = \mathbf{v}$.



The identity matrix is notated by $\mathbf{I}$ and looks like this: $\mathbf{I} =$

$$\begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

MATRIX-MATRIX MULTIPLICATION

Multiplying a $M \times N$ matrix with a $N \times P$ matrix results in a $M \times P$ matrix:

$$\mathbf{AB} = \underbrace{\begin{bmatrix} \sum_{i=1}^N A_{1i}B_{i1} & \sum_{i=1}^N A_{1i}B_{i2} & \dots & \sum_{i=1}^N A_{1i}B_{iP} \\ \sum_{i=1}^N A_{2i}B_{i1} & \sum_{i=1}^N A_{2i}B_{i2} & \dots & \sum_{i=1}^N A_{2i}B_{iP} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^N A_{Mi}B_{i1} & \sum_{i=1}^N A_{Mi}B_{i2} & \dots & \sum_{i=1}^N A_{Mi}B_{iP} \end{bmatrix}}_{(M \times P)} \quad \mathbf{A} = \underbrace{\begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix}}_{(M \times N)}$$

$$\mathbf{B} = \underbrace{\begin{bmatrix} B_{11} & B_{12} & \dots & B_{1P} \\ B_{21} & B_{22} & \dots & B_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ B_{N1} & B_{N2} & \dots & B_{NP} \end{bmatrix}}_{(N \times P)}$$

- Matrix-matrix multiplication is not always allowed! $\mathbf{A}$ and $\mathbf{B}$ can only be multiplied if $\mathbf{A}$ has as many columns as $\mathbf{B}$ has rows.
- Matrix-matrix multiplication depends on order! Except in special cases, $\mathbf{AB} \neq \mathbf{BA}$.

```

AB = np.dot(A, B)           # can use np.dot() for matrix-matrix multiplication too!
print(A.shape, "*", B.shape, "=", AB.shape)

```

$(4, 10) * (10, 6) = (4, 6)$

INTERPRETATION #1:

MATRIX-MATRIX MULTIPLICATION AS MATRIX- VECTOR MULTIPLICATION

Can apply all the same concepts from matrix-vector multiplication to matrix-matrix multiplication!

$$\mathbf{AB} = \mathbf{A} \begin{bmatrix} \mathbf{b}^{(1)} & \mathbf{b}^{(2)} & \cdots & \mathbf{b}^{(P)} \end{bmatrix} \\ = \begin{bmatrix} \mathbf{Ab}^{(1)} & \mathbf{Ab}^{(2)} & \cdots & \mathbf{Ab}^{(P)} \end{bmatrix}$$

```
AB = np.dot(A, B)    # matrix-matrix multiplication with np.dot()

m, n = A.shape
n, p = B.shape

AB_cols = np.zeros((m, p))

for i in range(p):
    AB_cols[:, i] = np.dot(A, B[:, i]) # multiply A with each column of B

print('using dot:\n', AB)
print('using matrix-vector products:\n', AB_cols)
```

```
using dot:
[[ 42  48  54]
 [114 136 158]]
matrix-vector products:
[[ 42.  48.  54.]
 [114  136  158]]
```

INTERPRETATION #2:

MATRIX-MATRIX MULTIPLICATION AS DOT PRODUCTS

AB measures overlap between rows of **A**
and columns of **B**

$$\mathbf{AB} = \begin{bmatrix} - & \mathbf{a}^{(1)T} & - \\ - & \mathbf{a}^{(2)T} & - \\ \vdots & \vdots & \vdots \\ - & \mathbf{a}^{(M)T} & - \end{bmatrix} \begin{bmatrix} | & | & \dots & | \\ \mathbf{b}^{(1)} & \mathbf{b}^{(2)} & \dots & \mathbf{b}^{(P)} \\ | & | & \dots & | \end{bmatrix}$$
$$= \begin{bmatrix} \mathbf{a}^{(1)} \cdot \mathbf{b}^{(1)} & \mathbf{a}^{(1)} \cdot \mathbf{b}^{(2)} & \dots & \mathbf{a}^{(1)} \cdot \mathbf{b}^{(P)} \\ \mathbf{a}^{(2)} \cdot \mathbf{b}^{(1)} & \mathbf{a}^{(2)} \cdot \mathbf{b}^{(2)} & \dots & \mathbf{a}^{(2)} \cdot \mathbf{b}^{(P)} \\ \vdots & \vdots & \dots & \vdots \\ \mathbf{a}^{(M)} \cdot \mathbf{b}^{(1)} & \mathbf{a}^{(M)} \cdot \mathbf{b}^{(2)} & \dots & \mathbf{a}^{(M)} \cdot \mathbf{b}^{(P)} \end{bmatrix}$$

```
AB = np.dot(A, B)    # matrix-matrix multiplication with np.dot()

m, n = A.shape
n, p = B.shape

AB_dots = np.zeros((m, p))

for i in range(m):  # loop over rows of A
    for j in range(p):  # loop over columns of B
        AB_dots[i, j] = np.dot(A[i, :], B[:, j])  # dot product

print('using dot:\n', AB)
print('using row-column dot products:\n', AB_dots)
```

```
using dot:
[[ 42  48  54]
 [114 136 158]]
using row-column dot products:
[[ 42.  48.  54.]
 [114. 136. 158.]]
```

SUMMARY

VECTORS

- a **vector** is a collection of real numbers, just like a **data point**!
- the size of a vector can be measured with a **vector norm**
- the similarity between two vectors can be measured with the **cosine similarity**, calculated with the **dot product**

MATRICES

- a **matrix** is an array of numbers, just like a **data set**!
- the **design matrix** contains a data point in each of its rows
- matrix-vector and matrix-matrix multiplication can be interpreted in terms of matrix **rows** and **columns**

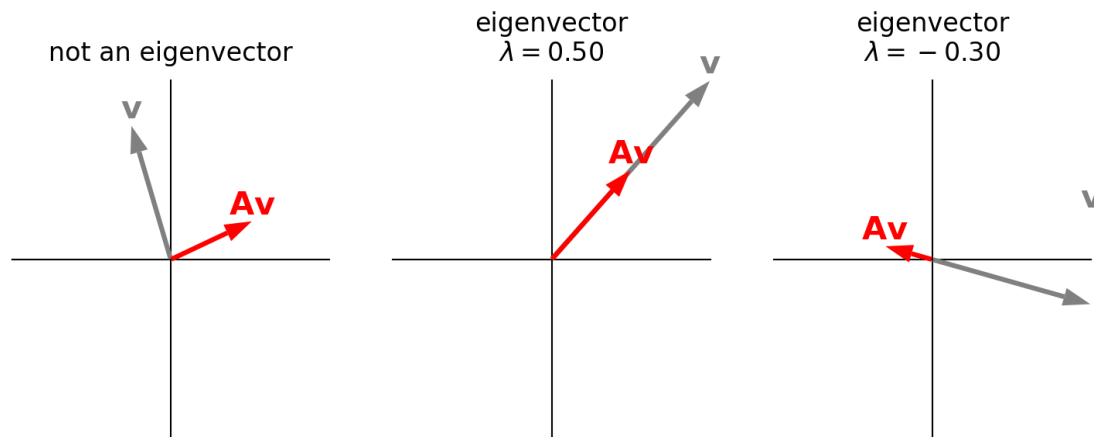
APPENDIX

EIGENVECTORS AND EIGENVALUES

An **eigenvector** of a **square** matrix is one that, when transformed/multiplied by the matrix, only gets re-scaled:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

The scaling factor λ is called its associated **eigenvalue**.



- An $M \times M$ matrix can have up to M linearly independent eigenvectors
- Not obvious why they are useful, but as you begin to do more linear algebra you'll see they come up all the time!

MATRIX INVERSE

If we transform a vector $\mathbf{v} \rightarrow \mathbf{A}\mathbf{v}$, can we then transform it back?

Yes -- use the **matrix inverse** of $\mathbf{A}$, notated by $\mathbf{A}^{-1}$:

$$\mathbf{A}^{-1}\mathbf{A}\mathbf{v} = \mathbf{v}$$

Note that $\mathbf{A}^{-1}\mathbf{A}$ is an identity transformation! So:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

Note: not all matrices are invertible!

- E.g. matrix of 0's transforms all vectors to the same thing (a vector of 0's), so you can't recover $\mathbf{v}$ from $\mathbf{A}\mathbf{v}$
- An $M \times M$ matrix is invertible only if all of its columns (or rows) are linearly independent